# Design and Implementation of a CI/CD Pipeline Using Jenkins, Docker, and AWS (ECR & EC2)

Shariqua Patel Farooque

# Automated Deployment of Order Service using Jenkins, Docker & AWS

## 1. Project Overview

This project implements a fully automated **Continuous Integration and Continuous Deployment (CI/CD) pipeline** for a containerized application. The solution integrates **GitHub, Jenkins, Docker, and AWS services (ECR & EC2)** to enable seamless application delivery from source code to production.

The pipeline is designed to:

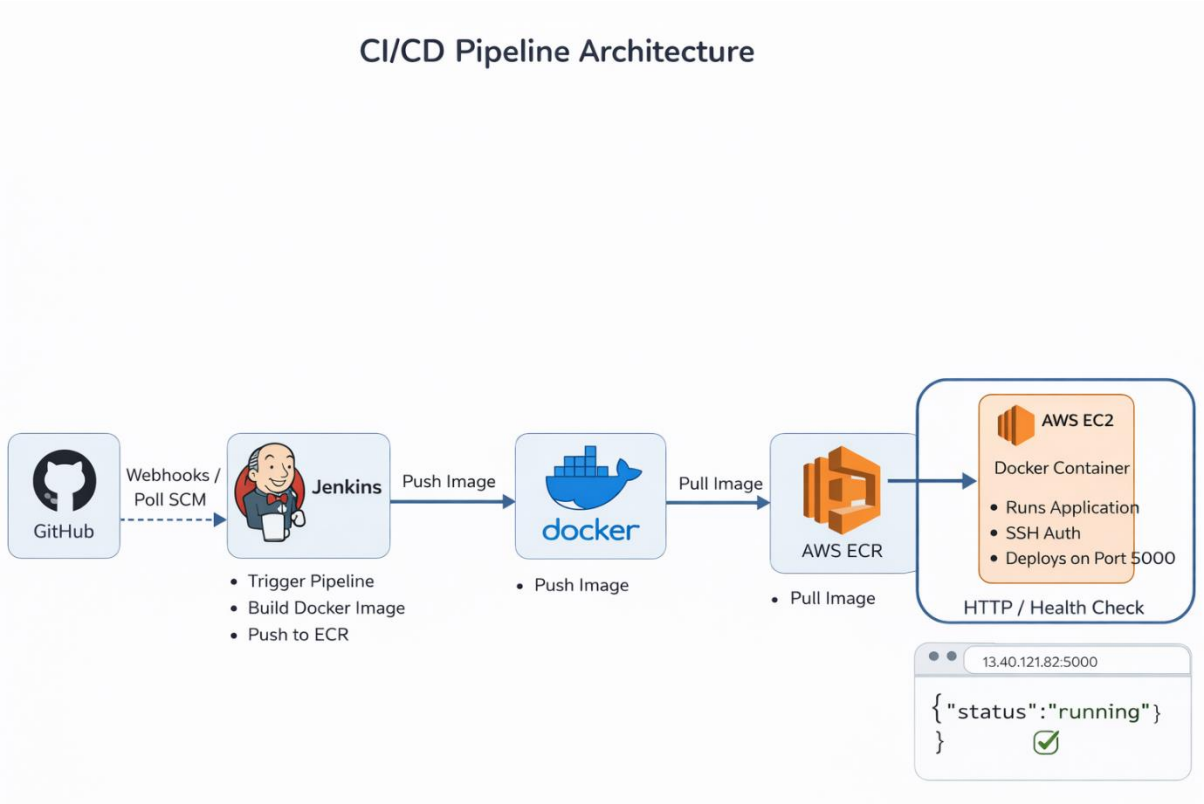
- Retrieve source code from a GitHub repository
- Build and package the application into a Docker image
- Store the image in Amazon Elastic Container Registry (ECR)
- Deploy the application to an Amazon EC2 instance
- Perform automated validation using a health check endpoint

## 2. Objectives

The primary objective of this project is to demonstrate practical implementation of DevOps concepts, including:

- Version control using Git and GitHub
- Continuous Integration using Jenkins
- Containerization using Docker
- Cloud-based deployment using AWS (EC2, ECR, IAM)
- Secure remote execution using SSH

## 3. System Architecture




GitHub → Jenkins → Docker → AWS ECR → EC2 → Application

**Workflow Description:**

1. The application source code is maintained in a GitHub repository.
2. Jenkins retrieves the latest version of the code.
3. A Docker image is built using the application source.
4. The image is tagged and pushed to Amazon ECR.
5. The EC2 instance pulls the latest image from ECR.
6. The existing container is replaced with a new instance.
7. The deployment is validated via an HTTP health check.

## 4. Technology Stack

| Layer | Technology |
|---|---|
| Operating System | Linux (Amazon EC2) |
| Version Control | Git, GitHub |
| CI/CD Tool | Jenkins |
| Containerization | Docker |
| Cloud Platform | AWS (EC2, ECR) |
| Security | IAM, SSH |
| CLI Tools | AWS CLI |

## 5. Source Code Management

The application source code is hosted on GitHub, enabling version control and collaborative development.

Jenkins integrates with GitHub to pull the latest code during pipeline execution.

**Example Git Workflow:**

```
git add .
git commit -m "Updated application"
git push origin main
```

This ensures that any updates to the repository can be integrated into the deployment pipeline.

## 6. CI/CD Pipeline Implementation

The pipeline consists of the following stages:

1. **Source Code Checkout**

   Jenkins retrieves the latest code from the GitHub repository.

2. **Docker Image Build**

   ```
   docker build -t order-service .
   ```

3. **Image Tagging**

   ```
   docker tag order-service:latest <ECR_URI>/order-service:latest
   ```

4. **Authentication with Amazon ECR**

   ```
   aws ecr get-login-password | docker login
   ```

**5. Image Push to ECR**

```
docker push <ECR_URI>/order-service:latest
```

**6. Deployment to EC2**

The following operations are executed on the EC2 instance:

```
docker pull <ECR_URI>/order-service:latest
docker stop order-service-container
docker rm order-service-container
docker run -d -p 5000:5000 order-service
```

**7. Health Check**

```
curl http://<EC2_IP>:5000
```

This confirms successful deployment and application availability.

## 7. AWS Services Utilized

**Amazon EC2**

- Hosts the application runtime environment
- Executes Docker containers

**Amazon ECR**

- Stores Docker images securely
- Provides version-controlled container registry

**AWS IAM**

- Manages access and permissions
- Enables secure interaction between services

## 8. Containerization Strategy

The application is containerized using Docker with:

- Base Image: Python 3.11 (slim)
- Exposed Port: 5000

This ensures portability, consistency, and scalability across environments.

## 9. Linux Operations

Deployment and verification tasks are performed using Linux commands:

docker ps
docker logs
docker pull

This demonstrates hands-on experience with Linux-based environments.

## 10. Security Considerations

- SSH key-based authentication is used for EC2 access
- IAM credentials are securely managed within Jenkins
- Docker images are stored in a private ECR repository

## 11. Deployment Verification

**Jenkins**

- Pipeline executed successfully without errors

**Amazon ECR**

- Image successfully pushed and versioned
- Pull timestamp confirms usage in deployment

**EC2 Instance**

docker ps

- Confirms container is running

**Application Endpoint**

http://13.40.121.82:5000

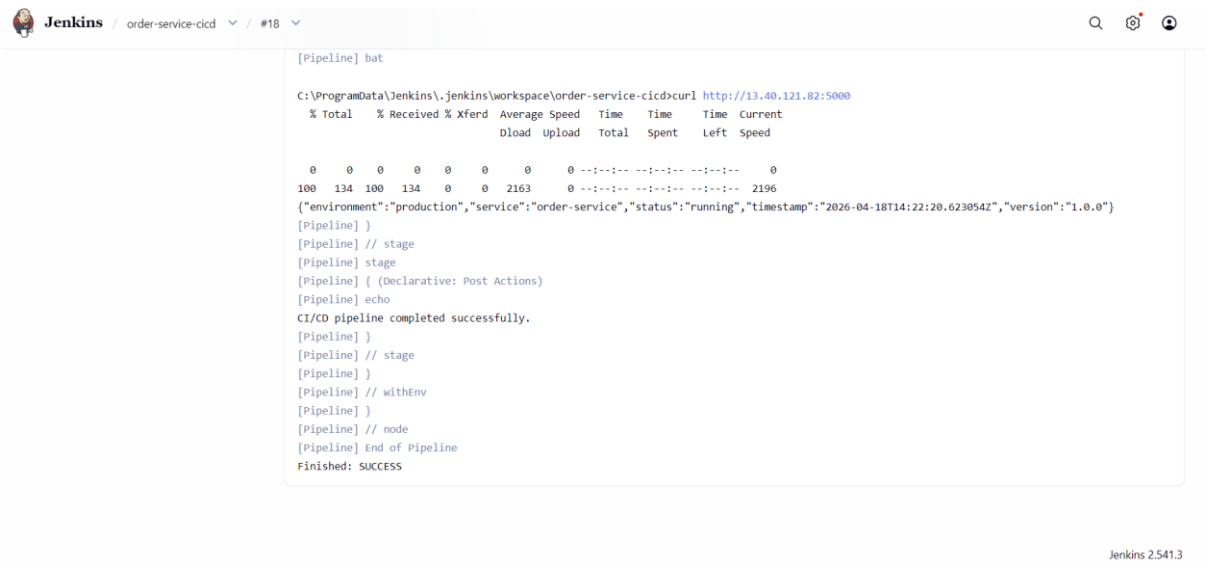
**Response:**


```
{
  "environment": "production",
  "service": "order-service",
  "status": "running",
  "version": "1.0.0"
}
```

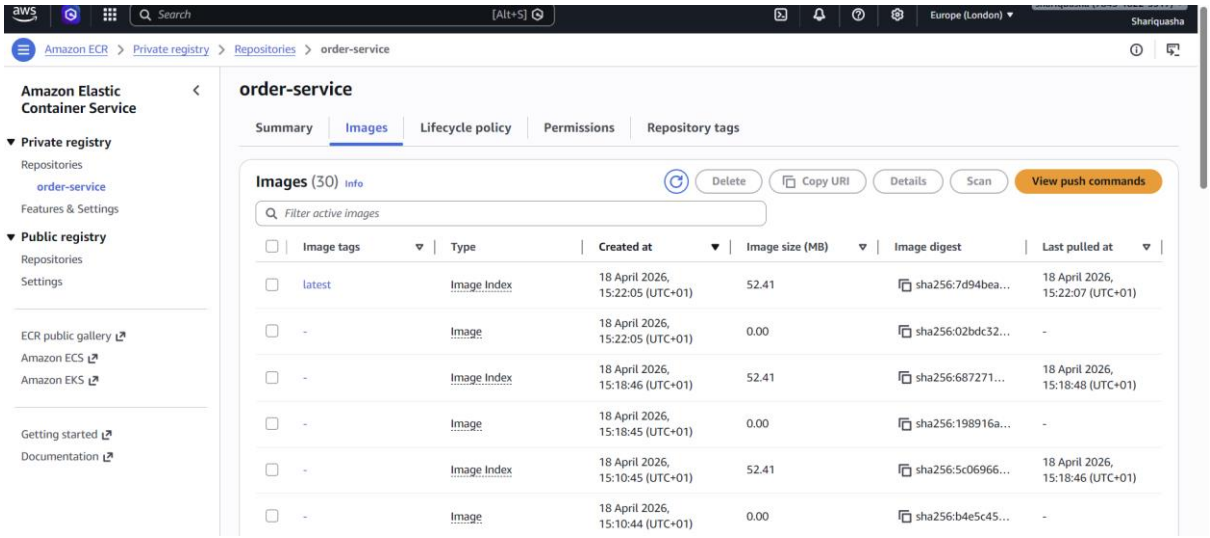

## 12. Evidence of Implementation

The following evidence supports successful implementation:

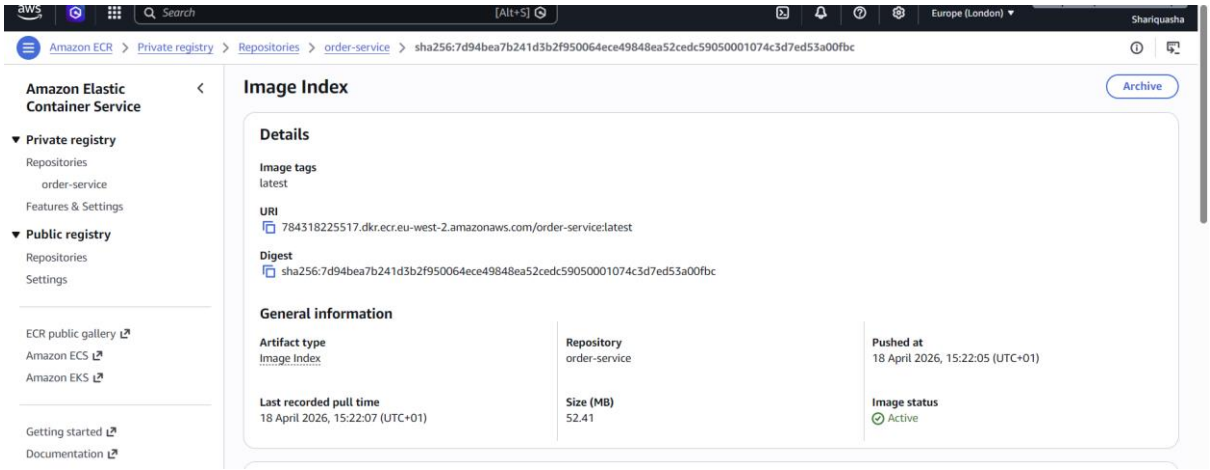
- Jenkins pipeline execution logs
- ECR repository and image details
- EC2 container status
- Application response via browser/Postman



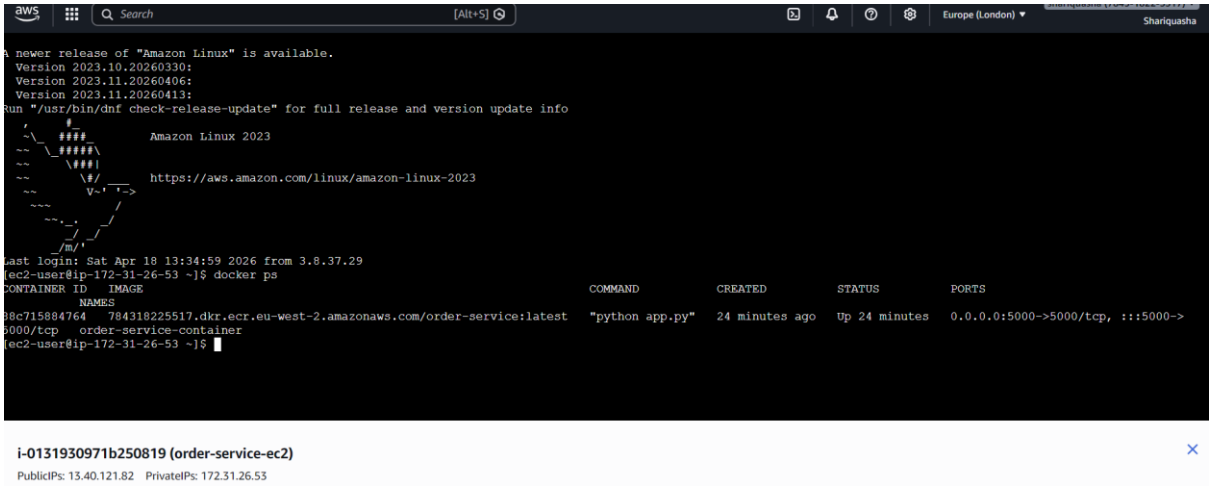

**Jenkins pipeline execution – Success**



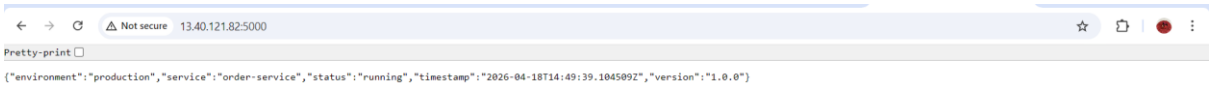

**Docker image in AWS ECR**




**Detailed view of Docker image in ECR showing URI, digest, and push timestamp**




**Docker container running on EC2**




**Application running successfully**

## 13. Challenges and Resolutions

### SSH Key Permission Issue

**Problem:** SSH key rejected due to insecure permissions
**Solution:** Resolved using Windows icacls to restrict access

### SSH Agent Plugin Compatibility Issue

**Problem:** SSH Agent plugin failure on Windows
**Solution:** Implemented sshUserPrivateKey for stable authentication

## 14. Future Enhancements

- Implement version-based image tagging
- Enable GitHub webhook-based triggers
- Configure custom domain using Route 53
- Implement HTTPS using Nginx and SSL certificates
- Integrate monitoring using AWS CloudWatch

## 15. Conclusion

This project successfully demonstrates the implementation of an end-to-end CI/CD pipeline, integrating modern DevOps tools and cloud technologies.

It highlights practical expertise in:

- Automated deployment workflows
- Containerized application delivery
- Cloud infrastructure management

The solution reflects real-world DevOps practices and scalable deployment strategies.